

Assignment 02

Diabetes Dataset — Cleaning, Imputation & Scaling

Abdulmanan

Roll No: 22i-2391

Muhammad Rohan

Roll No: 22i-2327

Department of Computer Science

1. Introduction

This assignment focuses on the essential data pre-processing pipeline applied to a real-world diabetes dataset. Raw datasets collected from medical records often contain missing values, inconsistent entries, and features at very different scales — all of which degrade the performance of machine learning models. Through this assignment we demonstrate how to systematically identify and handle these issues.

The three main stages covered are:

- Exploratory Data Analysis (EDA) — understanding the structure and quality of the raw data.
- Missing Value Imputation — replacing null/NaN entries using Mean and Median strategies.
- Feature Scaling — standardising numerical features so every column contributes equally.

Each stage is implemented in Python using industry-standard libraries: Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn.

2. Dataset Overview

The dataset used is `diabetes_unclean.csv` — a medical dataset containing patient records with various clinical measurements related to diabetes diagnosis. The columns include:

- ID — unique patient identifier
- No_Pation — patient number
- Gender — sex of the patient
- AGE — age of the patient
- Urea — blood urea level
- Cr — creatinine level
- HbA1c — glycated haemoglobin percentage (primary diabetes marker)
- Chol — cholesterol level
- TG — triglycerides
- HDL — high-density lipoprotein cholesterol
- LDL — low-density lipoprotein cholesterol
- VLDL — very-low-density lipoprotein cholesterol
- BMI — body mass index
- CLASS — diabetes classification label (Y/N/P)

The 'unclean' suffix in the filename signals that the dataset intentionally contains missing values and requires pre-processing before any analysis or modelling.

3. Methodology & Code Explanation

3.1 Importing Libraries

The first step is importing all required Python libraries. Each serves a specific purpose in the pipeline:

- pandas — tabular data manipulation (DataFrames, CSV I/O, descriptive statistics).
- matplotlib.pyplot — base plotting library for charts and figures.
- numpy — numerical computing, array operations, and statistical functions.
- seaborn — high-level statistical visualisation built on top of Matplotlib.

```
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
```

3.2 Loading the Dataset

The dataset is read from a CSV file using `pd.read_csv()`. The first 20 rows are displayed with `df.head(20)` to get an immediate visual impression of the data — column names, data types, and the presence of NaN values.

```
df = pd.read_csv("diabetes_unclean.csv")
df.head(20)
```

3.3 Descriptive Statistics

`df.describe()` generates a summary table of every numeric column. It shows the count (non-null observations), mean, standard deviation, minimum, the three quartiles (25th, 50th, 75th percentile), and maximum. This single call quickly surfaces anomalies such as impossible negative values or extreme outliers, and gives a feel for the scale of each feature.

```
df.describe()
```

3.4 Detecting Missing Values

Before any cleaning can be done, we need to know exactly where the gaps are. Two operations are used:

- `df.isnull().sum()` — returns a Series listing the number of missing entries in each column.

- `df.isnull().sum().sum()` — collapses the above to a single integer representing the total count of NaN values across the whole dataset.

```
df.isnull().sum()

print('Total Missing Values in Dataset: ', df.isnull().sum().sum())
```

3.5 Visualising Missing Values

A bar chart of missing values per column makes patterns immediately visible — which columns are most affected and whether the missingness is concentrated or spread across many features. The steps are:

- Filter to columns that actually have missing values: `missing_counts[missing_counts > 0]`.
- Sort in descending order so the worst columns appear on the left.
- Plot as a bar chart with a salmon colour fill, labelled axes, and a descriptive title.

```
missing_counts = df.isnull().sum()
plt.figure(figsize=(10, 6))
missing_counts[missing_counts > 0].sort_values(ascending=False).plot(
    kind='bar', color='salmon'
)
plt.title('Missing Values per Column')
plt.ylabel('Number of Missing Values')
plt.xlabel('Columns')
plt.show()
```

3.6 Filling Missing Values — Mean & Median Imputation

Two common single-value imputation strategies are applied and stored in separate DataFrames so they can be compared side-by-side:

- Mean Imputation (`df_Mean`) — every missing cell is replaced with the arithmetic mean of that column. This is appropriate when data is approximately normally distributed and there are no extreme outliers. The mean is sensitive to outliers, so an outlier-heavy column will produce a biased fill value.
- Median Imputation (`df_Median`) — every missing cell is replaced with the median (middle value) of that column. The median is robust to outliers and is preferred when the column contains skewed data or extreme values.

The `numeric_only=True` argument ensures that non-numeric columns (Gender, CLASS, etc.) are not involved in the mean/median calculation — those columns remain unchanged.

```
df_Mean = df.fillna(df.mean(numeric_only=True))
df_Median = df.fillna(df.median(numeric_only=True))

print('Total Missing Values in Mean Dataset: ', df_Mean.isnull().sum().sum())
print('Total Missing Values in Median Dataset: ',
      df_Median.isnull().sum().sum())
```

After running this code, both totals should print 0, confirming that all missing values have been successfully handled.

3.7 Dropping Non-Numeric Columns Before Scaling

StandardScaler only operates on numeric data. Columns that carry no continuous numerical information — ID, No_Pation (identifiers), Gender (categorical), AGE (discrete integer that will not be scaled here), and CLASS (the target label) — are dropped from df_Mean before the scaling step. Dropping them prevents type errors and ensures we only scale features that actually benefit from standardisation.

```
df_Mean = df_Mean.drop(
    ['ID', 'No_Pation', 'Gender', 'AGE', 'CLASS'],
    axis=1
)
```

3.8 Feature Scaling with StandardScaler

StandardScaler from Scikit-learn transforms each feature so that it has a mean of 0 and a standard deviation of 1. The formula applied column-by-column is:

$$z = (x - \mu) / \sigma$$

where μ is the column mean and σ is the column standard deviation.

Why scale?

- Many ML algorithms (k-NN, SVM, PCA, gradient descent-based models) are sensitive to the magnitude of feature values. Without scaling, a feature ranging from 0–300 (e.g. Chol) dominates a feature ranging from 0–5 (e.g. HbA1c).
- Standardisation brings all features to the same scale, giving each an equal opportunity to influence the model.

```

from sklearn.preprocessing import StandardScaler

scaler      = StandardScaler()
scaled_data = scaler.fit_transform(df_Mean)

df_scaled = pd.DataFrame(scaled_data, columns=df_Mean.columns)
df_scaled = df_scaled.round(4)
print(df_scaled.head())

```

`fit_transform()` first computes the mean and standard deviation of each column (fit), then applies the transformation (transform) in a single call. The result is wrapped back into a Pandas DataFrame with the original column names so it remains easy to inspect. Values are rounded to 4 decimal places for readability.

3.9 Dataset Visualisation — Tabular Snapshots

A helper function `plot_table()` renders the first 10 rows of any DataFrame as a formatted Matplotlib table. It is called three times to produce side-by-side visual evidence of the transformation at each stage:

- Before Cleaning — the raw df with NaN values visible.
- After Cleaning — `df_Mean` with all NaNs replaced by column means.
- After Scaling — `df_scaled` with zero-centred, unit-variance values.

```

def plot_table(df, title):
    fig, ax = plt.subplots(figsize=(12, 4))
    ax.axis('off')
    ax.set_title(title, fontweight='bold', fontsize=14)
    table = ax.table(
        cellText = df.head(10).values,
        colLabels = df.columns,
        cellLoc   = 'center',
        loc       = 'center'
    )
    table.auto_set_font_size(False)
    table.set_fontsize(10)
    table.auto_set_column_width(col=list(range(len(df.columns))))
    plt.show()

plot_table(df, "Before Cleaning")
plot_table(df_Mean, "After Cleaning")
plot_table(df_scaled, "After Scaling")

```

3.10 Data Summary After Pre-Processing

A final call to `df_scaled.describe()` confirms the effect of `StandardScaler`. In a perfectly scaled dataset every column should show a mean very close to 0 and a standard deviation very close to 1. Small floating-point deviations from these exact values are expected due to rounding.

```
df_scaled.describe()
```

4. Results & Discussion

The pre-processing pipeline was successfully applied to the diabetes dataset. The key observations at each stage are summarised below.

4.1 Raw Data

The raw dataset contained multiple columns with missing values. The bar chart produced in Section 3.5 clearly identified which columns were most affected. The `describe()` summary also revealed differing scales across features (e.g., Cholesterol and BMI have very different numeric ranges).

4.2 After Imputation

Both Mean and Median imputation eliminated all 0 NaN values from the dataset. The choice between them has practical consequences: columns with skewed distributions (such as clinical measurements that can spike for severely ill patients) benefit more from Median imputation because the median is not pulled by extreme values.

4.3 After Scaling

Post-standardisation, `df_scaled.describe()` confirmed that every numeric feature now has a mean ≈ 0 and a standard deviation ≈ 1 . This is the expected and correct outcome of `StandardScaler`. The dataset is now ready to be fed into distance-based or gradient-based machine learning models without any feature dominating due to its original scale.

5. Conclusion

This assignment demonstrated a complete data pre-processing workflow on a real medical dataset. Starting from a raw, incomplete CSV file we:

- Loaded and explored the data to understand its structure and quality.
- Detected and visualised missing values to guide imputation decisions.
- Applied both Mean and Median imputation strategies to fill all gaps.
- Dropped non-numeric and identifier columns that would interfere with scaling.

- Applied StandardScaler to produce a zero-mean, unit-variance feature set.
- Visualised before/after snapshots to verify each transformation step.

Clean, well-scaled data is the foundation of any reliable machine learning system. The techniques demonstrated here are directly applicable to any tabular dataset encountered in real-world data science projects.

Appendix: Complete Source Code

The following is the complete, commented Python code for the assignment. It is structured to run top-to-bottom in a Jupyter Notebook environment.

Cell 1 — Import Libraries

```
# Cell 1: Import all required libraries
import pandas as pd                # Data manipulation
import matplotlib.pyplot as plt    # Plotting
import numpy as np                 # Numerical operations
import seaborn as sns              # Statistical visualisation
```

Cell 2 — Load & Inspect Data

```
# Cell 2: Load the CSV and display first 20 rows
df = pd.read_csv("diabetes_unclean.csv")
df.head(20)           # Shows column names, types, and any visible NaN values
```

Cell 3 — Descriptive Statistics

```
# Cell 3: Generate descriptive statistics for all numeric columns
# Output: count, mean, std, min, 25%, 50%, 75%, max per column
df.describe()
```

Cell 4 — Detect Missing Values

```
# Cell 4a: Count missing values per column
df.isnull().sum()

# Cell 4b: Total missing values across the entire dataset
print('Total Missing Values in Dataset: ', df.isnull().sum().sum())
```

Cell 5 — Visualise Missing Values

```
# Cell 5: Bar chart of missing values — highlights the worst columns
missing_counts = df.isnull().sum()

plt.figure(figsize=(10, 6))
```

```
missing_counts[missing_counts > 0].sort_values(ascending=False).plot(
    kind='bar',
    color='salmon'
)
plt.title('Missing Values per Column')
plt.ylabel('Number of Missing Values')
plt.xlabel('Columns')
plt.show()
```

Cell 6 — Impute Missing Values

```
# Cell 6: Fill NaN values using Mean and Median strategies
# numeric_only=True ensures non-numeric columns are untouched
df_Mean = df.fillna(df.mean(numeric_only=True))
df_Median = df.fillna(df.median(numeric_only=True))

# Verify: both totals should now be 0
print('Total Missing Values in Mean Dataset: ', df_Mean.isnull().sum().sum())
print('Total Missing Values in Median Dataset: ',
      df_Median.isnull().sum().sum())
```

Cell 7 — Drop Non-Numeric Columns

```
# Cell 7: Remove identifier and categorical columns
# These columns cannot/should not be standardised
df_Mean = df_Mean.drop(
    ['ID', 'No_Pation', 'Gender', 'AGE', 'CLASS'],
    axis=1
)
```

Cell 8 — Import StandardScaler

```
# Cell 8: Import the StandardScaler from Scikit-learn
from sklearn.preprocessing import StandardScaler
```

Cell 9 — Apply Standard Scaling

```
# Cell 9: Fit and transform - z = (x - mean) / std for every column
scaler = StandardScaler()
scaled_data = scaler.fit_transform(df_Mean)

# Wrap back into a DataFrame so column names are preserved
df_scaled = pd.DataFrame(scaled_data, columns=df_Mean.columns)
```

```
df_scaled = df_scaled.round(4)    # Round for readability
print(df_scaled.head())
```

Cell 10 — Visualise Transformations

```
# Cell 10: Helper function to render a DataFrame as a Matplotlib table
def plot_table(df, title):
    fig, ax = plt.subplots(figsize=(12, 4))
    ax.axis('off')
    ax.set_title(title, fontweight='bold', fontsize=14)
    table = ax.table(
        cellText = df.head(10).values,
        colLabels = df.columns,
        cellLoc   = 'center',
        loc       = 'center'
    )
    table.auto_set_font_size(False)
    table.set_fontsize(10)
    table.auto_set_column_width(col=list(range(len(df.columns))))
    plt.show()

# Render three snapshots: raw → cleaned → scaled
plot_table(df, "Before Cleaning")
plot_table(df_Mean, "After Cleaning")
plot_table(df_scaled, "After Scaling")
```

Cell 11 — Final Summary Statistics

```
# Cell 11: Confirm scaling result
# Expected: mean ≈ 0, std ≈ 1 for every column
df_scaled.describe()
```